Lab 2257

From Streams to Insights: Building Real-Time Data Pipelines with Confluent and watsonx.data

Hands-on lab guide

Kavitha Paranthaman
Brand Technical Specialist (Data)
—
Kavitha.Paranthaman@ibm.com
309.826.5181

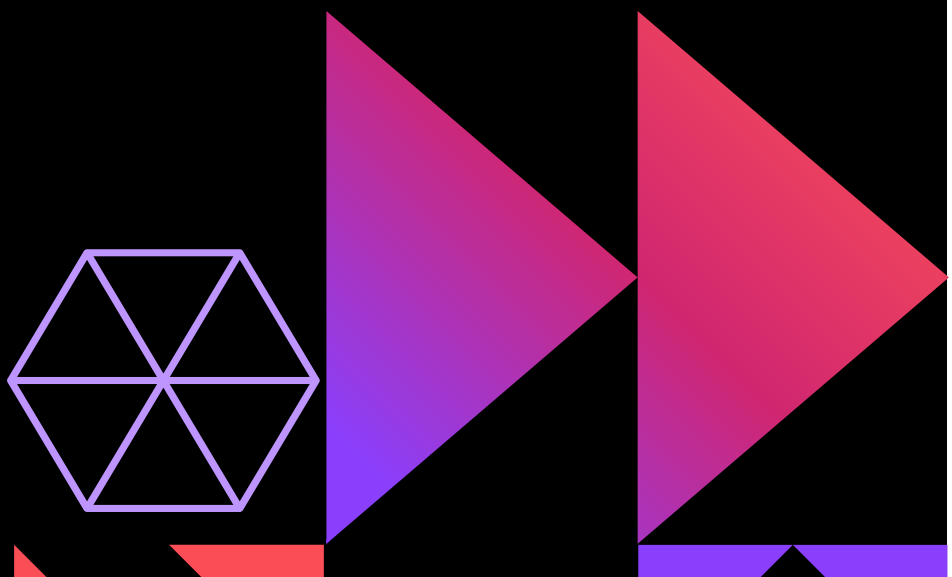


Table of Contents

1 Introduction 5

1.1 About this hands-on lab 6

2 Lab Environment and Access 7

Before starting the lab exercises, familiarize yourself with the environment that was provisioned for you. 7

2.1 Environment topology 7

2.2 Service endpoints 7

2.3 Reference data 7

3 Timing 8

4 Did not complete the lab? 8

5 Major task — Setting up the streaming pipeline 9

5.1 Architecture 9

5.2 Familiarization Walkthrough 10

5.3 Sub task — Exploring Kafka topics in Confluent Control Center 10

5.4 Sub task — Registering the Avro schema 11

6 Major task — Ingesting the stream into watsonx.data 12

6.1 Connecting watsonx.data to Confluent 12

6.2 Creating Iceberg tables in the fraud_analytics catalog 12

6.3 Running live queries with the Presto engine 13

7 Major task — Enriching transactions and detecting fraud 15

7.1 Joining streaming data with reference datasets 15

7.2 Writing suspicious transactions to a governed table 16

7.3 Reviewing the Fraud 360 dashboard 17

8 Feedback section 18

9 Getting help and troubleshooting 18

9.1 Reach out to the Lab instructors 18

9.2 Common troubleshooting tips 18

9.3 Getting help 18

10 Lab Reference Materials 19

10.1 IBM watsonx.data 19

10.2 Confluent Platform 7.6 19

10.3 Lab source code and automation 19

10.4 TechXchange 2026 19

Notices and disclaimers

© 2026 International Business Machines Corporation. No part of this document may be reproduced or transmitted in any form without written permission from IBM.

U.S. Government Users Restricted Rights — use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM.

This document is current as of the initial date of publication and may be changed by IBM at any time. Not all offerings are available in every country in which IBM operates.

Information in these presentations (including information relating to products that have not yet been announced by IBM) has been reviewed for accuracy as of the date of initial publication and could include unintentional technical or typographical errors. IBM shall have no responsibility to update this information.

This document is distributed “as is” without any warranty, either express or implied. In no event, shall IBM be liable for any damage arising from the use of this information, including but not limited to, loss of data, business interruption, loss of profit or loss of opportunity. IBM products and services are warranted per the terms and conditions of the agreements under which they are provided. The performance data and client examples cited are presented for illustrative purposes only. Actual performance results may vary depending on specific configurations and operating conditions.

IBM products are manufactured from new parts or new and used parts.
In some cases, a product may not be new and may have been previously installed. Regardless, our warranty terms apply.”

Any statements regarding IBM's future direction, intent or product plans are subject to change or withdrawal without notice.

Performance data contained herein was generally obtained in a controlled,
isolated environments. Customer examples are presented as illustrations of how those customers have used

IBM products and the results they may have achieved. Actual performance, cost, savings or other results in other operating environments may vary.

References in this document to IBM products, programs, or services does not imply that IBM intends to make such products, programs or services available in all countries in which IBM operates or does business.

Workshops, sessions and associated materials may have been prepared by independent session speakers, and do not necessarily reflect the views of IBM. All materials and discussions are provided for informational purposes only, and are neither intended to, nor shall constitute legal or other guidance or advice to any individual participant or their specific situation.

It is the customer's responsibility to ensure its own compliance with legal requirements and to obtain advice of competent legal counsel as to the identification and interpretation of any relevant laws and regulatory requirements that may affect the customer's business and any actions the customer may need to take to comply with such laws. IBM does not provide legal advice or represent or warrant that its services or products will ensure that the customer follows any law.

Notices and disclaimers (Continued)

Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products. IBM does not warrant the quality of any third-party products, or the ability of any such third-party products to interoperate with IBM's products. IBM expressly disclaims all warranties, expressed or implied, including but not limited to, the implied warranties of merchantability and fitness for a purpose.

The provision of the information contained herein is not intended to, and does not, grant any right or license under any IBM patents, copyrights, trademarks or other intellectual property right.

IBM, the IBM logo, and ibm.com are trademarks of International Business Machines Corporation, registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the Web at "Copyright and trademark information" at: www.ibm.com/legal/copytrade.shtml.

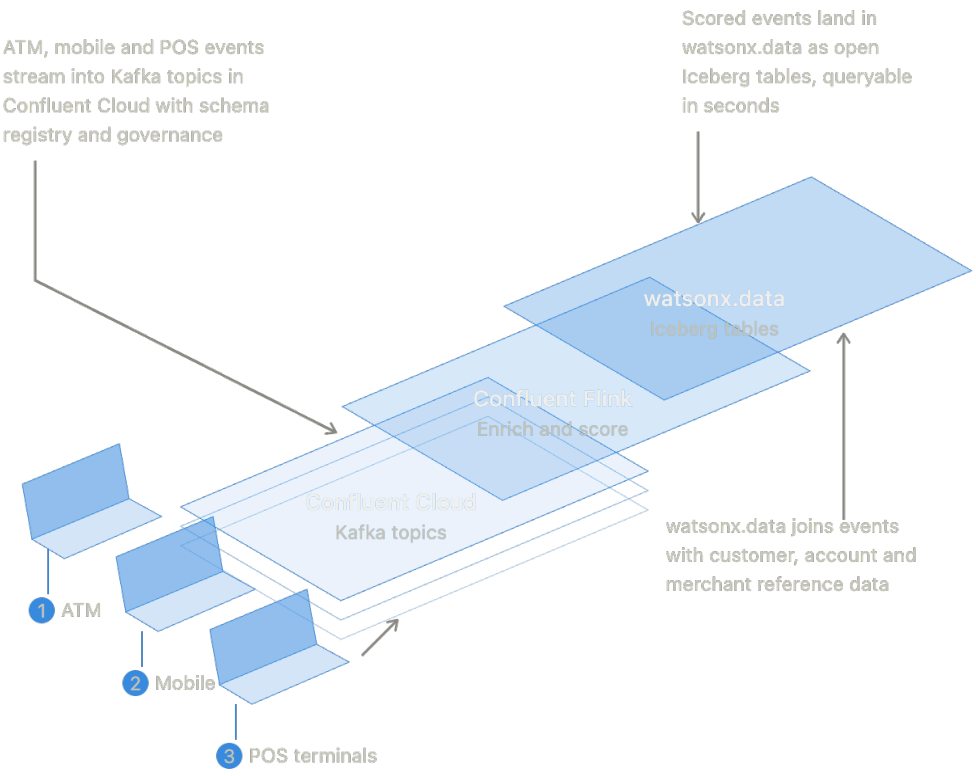
Use `IBM Plex Mono Light` for text that users must enter. This is consistent with IBM Style and avoids using quotation marks, which can be ambiguous as to whether they are meant to be included or

1 Introduction

IBM watsonx. data is a core component of watsonx, IBM's enterprise-ready AI and data platform that's designed to multiply the impact of AI across an enterprise.

The watsonx platform has three powerful components: the watsonx.ai studio for new foundation models, generative AI, and Machine Learning (traditional AI); the watsonx. data fit-for-purpose data lakehouse that provides the flexibility of a data lake with the performance of a data warehouse; and the watsonx. governance toolkit, which enables AI workflows that are built with responsibility, transparency, and explainability.

The watsonx. data component (the focus of this lab) makes it possible for enterprises to store, govern, query, and share data across their entire data estate from a single open lakehouse. Built on open formats and open table standards, watsonx.data allows multiple query engines to operate against one copy of data without duplication or vendor lock-in. With watsonx.data, enterprises can pair the low-cost storage economics of a data lake with warehouse-class query performance, connect to existing sources rather than migrating wholesale, and bring streaming and at-rest data into the same governed environment to reduce cost, cut latency, and accelerate the time between an event occurring and an insight being available.



Confluent extends that architecture to data in motion. Built on Apache Kafka, Confluent provides the event streaming backbone that carries continuous transactional activity across an enterprise, along with the connectors, schema management, and stream processing needed to make those events consumable downstream. Together, the two platforms close the gap between operational events and analytical insight.

That gap is a persistent challenge in financial services. Institutions generate continuous streams of transactional data, yet integrating those streams into a unified analytics platform remains difficult, and the delay between an event occurring and an analyst being able to query it directly limits use cases like fraud detection.

This lab addresses that problem hands-on. You will connect a Confluent streaming source to watsonx. data, ingest live transaction data, enrich it, and join it with existing enterprise datasets to produce governed, queryable results. By the end, you will understand how to connect streaming pipelines to a lakehouse, how watsonx. data reduces integration effort and latency, and how to apply that pattern to real-time use cases in your own environment.

1.1 About this hands-on lab

The main objective of this hands-on lab is to present an end-to-end view of what it takes to move transactional data from a live stream into a governed lakehouse where it can be queried. Having seen many streaming integration projects undertaken, it's clear that the most common route to success is having a clear picture of the questions the business needs answered before the pipeline is built, because those questions determine the schema, the enrichment logic, and how the streaming data needs to join to what already exists in the warehouse. Too often, projects stall because streaming is treated as a plumbing exercise and the analytics side is designed after the fact. Real-time data is powerful, but a stream that lands in a format nobody can join to is not an asset. Solution designers need to be practical about the shape of the data they are landing, and to that end, this hands-on lab provides insights on how to architect a Confluent-to-watsonx.data pipeline that supports the analytical use cases the business is asking for.

By the end of this lab, you will be able to:

- Configure a Confluent topic as a streaming source and inspect the transaction events flowing through it
- Ingest that stream into watsonx.data and register it as a queryable Iceberg table
- Enrich incoming transaction records and resolve them against reference data (customer profiles, merchant risk)
- Join streaming data with existing enterprise datasets in a single Presto SQL query
- Apply governance and access controls so the resulting tables are safe to expose to analysts
- Explain how this pattern supports a fraud detection use case and where its latency limits sit

The lab runs on a dedicated IBM Cloud VSI (Virtual Server Instance) running RHEL 9 in the ca-tor (Toronto) region, provisioned through IBM TechZone. Confluent Platform 7.6.1 (including Kafka, Schema Registry, Control Center, ksqlDB, and Flink) is pre-installed and started automatically. The watsonx.data lakehouse is provisioned as a shared SaaS instance; connection details are provided on your TechZone reservation page.

2 Lab Environment and Access

Before starting the lab exercises, familiarize yourself with the environment that was provisioned for you.

2.1 Environment topology

Each participant receives a dedicated IBM Cloud VSI with the following components pre-installed and running. All services are accessible on the VSI's public IP address, which is shown on your TechZone reservation page.

Architecture data flow:

```
Synthetic Producer  →  Kafka (card_transactions)  →  ksqlDB enrichment
→  Kafka (enriched_transactions / suspicious_transactions)
→  Apache Iceberg on IBM Cloud Object Storage
→  watsonx.data Presto Engine  →  Analyst queries / Fraud 360 Dashboard
```

2.2 Service endpoints

The following services are running on your VSI. Replace <YOUR_VM_IP> with the IP shown on your reservation page.

Service	URL / Address	Notes
Confluent Control Center	http://<YOUR_VM_IP>:9021	Web UI — browse topics, schemas, consumers
Kafka Bootstrap Server	<YOUR_VM_IP>:9092	Used by producers and consumers
Schema Registry	http://localhost:8081	Avro schema management REST API
Flink REST API	http://localhost:8082	Stream processing job management
Confluent ksqlDB	http://localhost:8088	Continuous SQL enrichment streams

> 📝 **NOTE:** watsonx.data credentials (Presto host, API key) are provided on your TechZone reservation page under 'Reservation Details'. You will need them in Section 6.

2.3 Reference data

Three reference datasets are pre-loaded on your VSI in the ~/data/ directory. They will be used in Section 7 to enrich streaming transactions.

File	Key columns	Purpose
customer_profiles.csv	customer_id, card_id, risk_tier, home_city, avg_monthly_txn, account_status	Customer risk classification and baseline spending
account_master.csv	card_id, account_id, card_type, credit_limit, current_balance, account_status	Card account limits and balances

merchant_reference.csv	merchant_name, merchant_category, risk_level, avg_txn_amount, country	Merchant risk classification

BM TechXchange 2026 / © 2026 IBM Corporation

3 Timing

All TechXchange labs are created to last for 90 minutes. The recommended pacing is shown below. Limit lab activities to what participants can achieve in 75 minutes, and provide ample references for further learning.

Section	Task	Recommended time
2	Environment familiarization and service verification	10 min
5	Setting up the streaming pipeline	20 min
6	Ingesting the stream into watsonx.data	25 min
7	Enrichment, fraud detection, and dashboard	25 min
8–10	Feedback and wrap-up	10 min

Walk students through a key feature of the product, use diagrams and screenshots where appropriate, and keep instructions short — no more than 2–3 sentences per step.

4 Did not complete the lab?

If you did not finish the lab during the session, you can continue in the self-paced lab room. Your TechZone reservation is valid for 2 days after the conference session ends. To resume:

1. Navigate to https://techzone.ibm.com and log in with your IBM ID.
2. Under My Reservations, locate 'Lab 2257 — Real-Time Fraud Detection'.
3. Click Open to re-open the reservation details and retrieve your VM IP and watsonx.data credentials.
4. SSH into your VM and run ~/verify_lab_setup.sh to confirm all services are still running.
5. Continue from the section you left off.

> 💡 **TIP:** If your reservation has expired, raise a support ticket at https://techzone.ibm.com/support and reference Lab 2257.

IBM TechXchange 2026 / © 2026 IBM Corporation

5 Major task — Setting up the streaming pipeline

In this section you will verify that the Confluent Platform services are running correctly, explore the synthetic transaction stream, and confirm that the Avro schema is registered. These are the foundation for everything that follows.

5.1 Architecture

The lab implements a streaming-to-lakehouse architecture. The diagram below shows how data flows from the synthetic producer to the Presto query engine.

> 📝 **NOTE:** Use tools like https://diagrams.net to create architecture diagrams for your own labs. Export as PNG or JPG and insert into Word using Insert → Pictures.

Component descriptions:

- Synthetic Transaction Producer — a Python script (transaction_producer.py) generates credit-card events at ~2–6 events/second across 50 simulated customers (CUST_1001–CUST_1050) and 5 merchants (Amazon, Walmart, Apple Store, Luxury Watches Direct, CryptoExchange Fast). ~8 % of events are flagged suspicious (amount > $1,500 or high-risk merchant). Channels: POS_CHIP, POS_CONTACTLESS, ONLINE, ATM.
- Kafka Topic: card_transactions — 3 partitions, replication factor 1, 24-hour retention. Schema: transaction_id, card_number, customer_id, amount, currency, merchant_id, merchant_name, merchant_category, channel, city, country, latitude, longitude, timestamp.
- Kafka Topic: enriched_transactions — 3 partitions. Populated by ksqlDB enrichment stream that joins card_transactions with merchant reference data and computes a fraud_score (0.0–1.0).
- Kafka Topic: suspicious_transactions — 1 partition. Contains only events where fraud_score > 0.75, written by a second ksqlDB filter stream.
- Apache Iceberg on IBM Cloud Object Storage (ca-tor) — three Iceberg tables in catalog iceberg_data, schema fraud_analytics: transactions_raw, enriched_transactions, suspicious_transactions. Partitioned by day(event_timestamp). Format: Parquet.
- Presto Engine (watsonx.data) — executes ad-hoc and scheduled SQL queries across all Iceberg tables. Supports cross-table joins including reference CSVs. Typical query latency: 62–145 ms.

Lab Architecture — service interconnectivity:

Component	Host	Port	Protocol
Kafka Broker	VSI (RHEL 9, ca-tor)	9092	PLAINTEXT
Schema Registry	VSI (RHEL 9, ca-tor)	8081	HTTP
Confluent Control Center	VSI (RHEL 9, ca-tor)	9021	HTTP
ksqlDB	VSI (RHEL 9, ca-tor)	8088	HTTP
Flink REST API	VSI (RHEL 9, ca-tor)	8082	HTTP
watsonx.data Presto	IBM Cloud SaaS	8443	HTTPS

IBM Cloud Object Storage	IBM Cloud SaaS (COS)	443	HTTPS S3

IBM TechXchange 2026 / © 2026 IBM Corporation

5.2 Familiarization Walkthrough

Since lab environments are new and time is limited, follow these steps to walk through the environment before running queries.

6. SSH into your VSI using the credentials shown on your TechZone reservation page:

```
ssh labuser@<YOUR_VM_IP>
```

7. Run the lab verification script to confirm all services are healthy:

```
~/verify_lab_setup.sh
```

Expected output (all green):

```
[PASS] Kafka broker is reachable on port 9092
[PASS] Schema Registry is healthy (HTTP 200)
[PASS] Control Center is reachable on port 9021
[PASS] Topic exists: card_transactions
[PASS] Topic exists: enriched_transactions
[PASS] Topic exists: suspicious_transactions
[PASS] Avro schema registered: card_transactions-value
[PASS] transaction-producer systemd service is running
[PASS] Flink REST API is healthy
```

> 💡 **TIP:** If any check shows [FAIL], refer to Section 9.2 for common troubleshooting steps before proceeding.

8. Open Confluent Control Center in your browser:

```
http://<YOUR_VM_IP>:9021
```

9. Click on the cluster → Topics to see the three topics: card_transactions, enriched_transactions, suspicious_transactions.
10. Click card_transactions → Messages to watch live transaction events flowing through the topic.

5.3 Sub task — Exploring Kafka topics in Confluent Control Center

Control Center provides a real-time view of your Kafka cluster. In this sub-task you will inspect the message format and throughput of the card_transactions topic.

11. In Control Center, navigate to Topics → card_transactions → Messages.
12. Set the offset to 'Latest' and observe the incoming events. Each message looks like:

```
{
  "transaction_id": "3fa2e1b0-c14d-4e87-9b22-7f4a1d3e90cc",
  "card_number":    "CARD_4111_1023",
  "customer_id":    "CUST_1023",
  "amount":          2847.50,
  "currency":       "USD",
  "merchant_id":    "M_CRYP_05",
  "merchant_name":  "CryptoExchange Fast",
  "merchant_category": "Financial Services",
```

```
  "channel":        "ONLINE",
  "city":           "Singapore",
  "country":        "Singapore",
  "latitude":        1.3521,
  "longitude":       103.8198,
  "timestamp":      "2026-09-04 14:22:07"
}
```

13. Navigate to Topics → card_transactions → Overview and note the current message rate (typically ~280–420 msg/min given the producer's 0.15–0.45 s sleep interval).
14. Click Consumer groups to verify that the ksqlDB enrichment stream is consuming from the topic.

5.4 Sub task — Registering the Avro schema

The Avro schema for card_transactions-value is registered automatically by the post-deploy playbook. In this sub-task you will verify the registration and inspect the schema fields.

15. Open a terminal on your VSI and query the Schema Registry:

```
curl http://localhost:8081/subjects
```

Expected response:

```
["card_transactions-value"]
```

16. Retrieve the full schema definition:

```
curl http://localhost:8081/subjects/card_transactions-value/versions/latest | python3 -
m json.tool
```

17. In Control Center, navigate to Schema Registry to view the schema visually and confirm all 14 fields are present.

> 💡 **TIP:** If the schema is missing, the ksqlDB CREATE STREAM statement in Section 6 will fail with a serialization error. Re-run verify_lab_setup.sh and check: journalctl -u transaction-producer -n 50

IBM TechXchange 2026 / © 2026 IBM Corporation

6 Major task — Ingesting the stream into watsonx.data

In this section you will connect watsonx.data to the Confluent Kafka cluster, create Iceberg tables in the fraud_analytics catalog, and verify that streaming data is landing in queryable form.

6.1 Connecting watsonx.data to Confluent

watsonx.data uses a Kafka connector (via the Presto Kafka connector plugin) to read from Confluent topics. You will configure the connection using the wxdata-connection.json template on your VSI.

18. Retrieve your watsonx.data Presto host and API key from your TechZone reservation page. They appear under 'Reservation Details → watsonx.data credentials'.
19. On your VSI, view the connection configuration template:

```
cat ~/wxdata-connection.json
```

20. The template looks like this — update the two placeholder values:

```
{
  "connection_name": "watsonx-data-fraud-lab",
  "description":     "watsonx.data Presto connection for Real-Time Fraud Detection
Lab",
  "type":            "presto",
  "host":            "<wxd-host-from-reservation>",   ← replace with your Presto host
  "port":            8443,
  "username":        "ibmlhapikey",
  "password":        "<your-api-key>",                ← replace with your watsonx.data
API key
  "ssl":             true,
  "catalog":         "iceberg_data",
  "schema":          "fraud_analytics"
}
```

21. In the watsonx.data web UI, navigate to Infrastructure manager → Add component → Connection and paste the updated JSON. Click Test connection to verify.

> 💡 **TIP:** If the test fails with SSL handshake error, ensure port 8443 is reachable outbound from your VSI. Run: curl -k https://<wxd-host>:8443 to test connectivity.

6.2 Creating Iceberg tables in the fraud_analytics catalog

With the connection established, you will create three Iceberg tables in the iceberg_data catalog. The tables are backed by IBM Cloud Object Storage (COS) in the ca-tor region.

22. In the watsonx.data SQL editor (Data Manager → SQL), create the schema:

```
CREATE SCHEMA IF NOT EXISTS iceberg_data.fraud_analytics
WITH (location = 's3a://<your-cos-bucket>/fraud-lab/');
```

> 📝 **NOTE:** Replace <your-cos-bucket> with the COS bucket name shown on your TechZone reservation page.

23. Create the raw transactions table:

```
CREATE TABLE IF NOT EXISTS iceberg_data.fraud_analytics.transactions_raw (
    transaction_id    VARCHAR,
    card_number       VARCHAR,
```

```
    customer_id       VARCHAR,
    amount            DOUBLE,
    currency          VARCHAR,
    merchant_id       VARCHAR,
    merchant_name     VARCHAR,
    merchant_category VARCHAR,
    channel           VARCHAR,
    city              VARCHAR,
    country           VARCHAR,
    latitude          DOUBLE,
    longitude         DOUBLE,
    event_timestamp   TIMESTAMP
)
WITH (format = 'PARQUET',
      partitioning = ARRAY['day(event_timestamp)']);
```

24. Create the enriched_transactions table (same DDL, add fraud_score and risk_tier columns):

```
CREATE TABLE IF NOT EXISTS iceberg_data.fraud_analytics.enriched_transactions (
    transaction_id    VARCHAR,
    card_number       VARCHAR,
    customer_id       VARCHAR,
    amount            DOUBLE,
    currency          VARCHAR,
    merchant_id       VARCHAR,
    merchant_name     VARCHAR,
    merchant_category VARCHAR,
    channel           VARCHAR,
    city              VARCHAR,
    country           VARCHAR,
    fraud_score       DOUBLE,
    risk_tier         VARCHAR,
    event_timestamp   TIMESTAMP
)
WITH (format = 'PARQUET',
      partitioning = ARRAY['day(event_timestamp)']);
```

25. Create the suspicious_transactions table (same structure as enriched_transactions).
26. Verify the tables were created:

```
SHOW TABLES IN iceberg_data.fraud_analytics;
```

Expected output:

```
  Table
------------------------
 enriched_transactions
 suspicious_transactions
 transactions_raw
```

6.3 Running live queries with the Presto engine

The ksqlDB sink connector continuously writes enriched events from Confluent into the Iceberg tables. You will now query the data in near-real time.

27. Run a row count to see how many events have landed:

```
SELECT COUNT(*) AS total_events
FROM iceberg_data.fraud_analytics.transactions_raw;
```

28. **23.** Run a live sampling query to inspect the most recent events:

```
SELECT transaction_id, customer_id, merchant_name,
       amount, channel, event_timestamp
FROM   iceberg_data.fraud_analytics.transactions_raw
ORDER  BY event_timestamp DESC
LIMIT  20;
```

29. **24.** Check the distribution of transaction channels:

```
SELECT channel,
       COUNT(*)              AS cnt,
       ROUND(AVG(amount), 2) AS avg_amount
FROM   iceberg_data.fraud_analytics.transactions_raw
GROUP  BY channel
ORDER  BY cnt DESC;
```

> 💡 **TIP:** Re-run the COUNT(*) query every 30 seconds to watch the row count increase as new events land. Typical Confluent-to-Iceberg ingestion lag is < 30 seconds.

IBM TechXchange 2026 / © 2026 IBM Corporation

7 Major task — Enriching transactions and detecting fraud

In this section you will join the streaming transaction data with the enterprise reference datasets, compute a fraud score, write high-risk transactions to a governed table, and review the results in the Fraud 360 dashboard.

7.1 Joining streaming data with reference datasets

First, upload the reference CSV files from your VSI to COS so that watsonx.data can read them as external tables alongside the Iceberg streaming tables.

30. On your VSI, list the reference data files:

```
ls -lh ~/data/
```

You should see:

```
customer_profiles.csv   — 50+ customers: customer_id, card_id, risk_tier, home_city,
avg_monthly_txn
account_master.csv      — 50+ accounts:  card_id, account_id, card_type, credit_limit,
current_balance
merchant_reference.csv  — merchants:     merchant_name, merchant_category, risk_level,
avg_txn_amount
```

31. Upload the reference files to your COS bucket (use the ibmcloud CLI or COS web UI):

```
ibmcloud cos upload \
  --bucket <your-cos-bucket> \
  --key    fraud-lab/reference/customer_profiles.csv \
  --file   ~/data/customer_profiles.csv

ibmcloud cos upload \
  --bucket <your-cos-bucket> \
  --key    fraud-lab/reference/account_master.csv \
  --file   ~/data/account_master.csv

ibmcloud cos upload \
  --bucket <your-cos-bucket> \
  --key    fraud-lab/reference/merchant_reference.csv \
  --file   ~/data/merchant_reference.csv
```

32. Register the reference CSVs as external tables in the watsonx.data SQL editor:

```
CREATE TABLE iceberg_data.fraud_analytics.customer_profiles (
    customer_id      VARCHAR,
    card_id          VARCHAR,
    risk_tier        VARCHAR,
    home_city        VARCHAR,
    avg_monthly_txn  DOUBLE,
    account_status   VARCHAR
)
WITH (format = 'CSV',
      external_location = 's3a://<your-bucket>/fraud-
lab/reference/customer_profiles.csv',
      skip_header_line_count = 1);
```

33. Run the enriched three-way JOIN to combine streaming transactions with customer and merchant data:

```
SELECT
    t.transaction_id,
    t.customer_id,
    c.risk_tier,
    c.home_city,
    t.merchant_name,
    m.risk_level                                          AS merchant_risk,
    t.amount,
    a.credit_limit,
    ROUND(t.amount / NULLIF(a.credit_limit, 0) * 100, 1)       AS pct_of_limit,
    t.channel,
    t.event_timestamp
FROM   iceberg_data.fraud_analytics.transactions_raw       t
LEFT JOIN iceberg_data.fraud_analytics.customer_profiles   c  ON t.customer_id  =
c.customer_id
LEFT JOIN iceberg_data.fraud_analytics.account_master      a  ON t.card_number  =
a.card_id
LEFT JOIN iceberg_data.fraud_analytics.merchant_reference  m  ON t.merchant_name =
m.merchant_name
ORDER  BY t.event_timestamp DESC
LIMIT  50;
```

7.2 Writing suspicious transactions to a governed table

A transaction is classified as suspicious when the fraud_score exceeds 0.75. The ksqlDB stream populates the suspicious_transactions Iceberg table automatically. You will now query it and compute aggregate fraud metrics.

34. Count total suspicious transactions and total value at risk:

```
SELECT COUNT(*)               AS suspicious_count,
       ROUND(SUM(amount), 2)  AS total_value_at_risk
FROM   iceberg_data.fraud_analytics.suspicious_transactions;
```

35. Identify the highest-risk merchant categories:

```
SELECT merchant_category,
       COUNT(*)                    AS incident_count,
       ROUND(AVG(amount), 2)       AS avg_fraudulent_amount,
       ROUND(MAX(amount), 2)       AS max_amount
FROM   iceberg_data.fraud_analytics.suspicious_transactions
GROUP  BY merchant_category
ORDER  BY incident_count DESC;
```

36. Find customers with multiple suspicious transactions in the last hour:

```
SELECT customer_id,
       COUNT(*)       AS incident_count,
       SUM(amount)    AS exposure
FROM   iceberg_data.fraud_analytics.suspicious_transactions
WHERE  event_timestamp >= NOW() - INTERVAL '1' HOUR
GROUP  BY customer_id
HAVING COUNT(*) > 1
ORDER  BY exposure DESC;
```

> 💡 **TIP:** The suspicious_transactions table is partitioned by day(event_timestamp). Always include a time predicate in your WHERE clause for best Presto query performance.

7.3 Reviewing the Fraud 360 dashboard

The Fraud 360 Command Center dashboard (fraud-detection-dashboard.html) visualizes the live pipeline metrics. It simulates the Presto query output you just ran, showing the forensic join across Kafka topics and Iceberg tables.

37. Open fraud-detection-dashboard.html in your browser. The file is linked from the lab materials URL or can be opened directly from the lab repo.
38. Observe the four KPI cards at the top:

- Processed Events — total events flowing through the card_transactions Kafka topic
- Fraud Incidents Blocked — count of CRITICAL_LIMIT_BREACH and HIGH_RISK_STORE classifications
- Total Value at Risk — cumulative USD amount of blocked transactions in the last 60 minutes
- Presto Engine Query Latency — simulated distributed Iceberg join latency (62–145 ms)

39. Review the Live Fraud 360 Forensic Table — this shows the enriched join output with fraud classification badges (CRITICAL, HIGH, WARNING, AUTHORIZED) and a risk score bar (0–100) matching the ksqlDB fraud_score field.
40. Observe the architecture flow bar at the top of the dashboard, which mirrors the pipeline you built: Confluent Datagen → Kafka (card_transactions) → ksqlDB Anomaly Stream → Apache Iceberg (iceberg_data) → watsonx.data Presto Engine.

> 📝 **NOTE:** The dashboard is a static HTML simulation for demonstration purposes. In production it would be connected to live Presto query results via a BI tool such as IBM Cognos Analytics, Tableau, or a custom Python/Streamlit application pointing to the watsonx.data Presto endpoint.

IBM TechXchange 2026 / © 2026 IBM Corporation

8 Feedback section

Your feedback helps improve this lab for future participants. Please take 2 minutes to complete the survey before leaving the session.

41. Scan the QR code on the room screen, or navigate to the TechXchange 2026 feedback portal.
42. Select Lab 2257 — From Streams to Insights.
43. Rate the lab on clarity, pacing, and technical depth, and leave any written comments.

Alternatively, email feedback directly to the lab author: Kavitha.Paranthaman@ibm.com with subject line 'Lab 2257 Feedback'.

9 Getting help and troubleshooting

This section provides information about getting help with your lab and common troubleshooting topics.

9.1 Reach out to the Lab instructors

During the session, raise your hand or approach the instructor at the front of the room. For self-paced participants, email Kavitha.Paranthaman@ibm.com.

9.2 Common troubleshooting tips

Use this section to resolve common issues before raising a support ticket.

Symptom	Likely cause	Fix
verify_lab_setup.sh shows [FAIL] for Kafka broker	Confluent Kafka service stopped	sudo systemctl start confluent-kafka && sleep 10 && ~/verify_lab_setup.sh
Control Center shows no messages on card_transactions	transaction-producer service stopped	sudo systemctl restart transaction-producer
Schema Registry returns HTTP 000 or connection refused	confluent-schema-registry not started	sudo systemctl start confluent-schema-registry
watsonx.data Test connection fails	Wrong Presto host or API key; trailing spaces	Re-copy credentials from TechZone reservation page (no trailing spaces)
Presto query on transactions_raw returns 0 rows	Kafka sink connector to Iceberg not running	In Control Center → Connectors, check status; restart the IcebergSinkConnector
CREATE SCHEMA fails with 'bucket not found'	COS bucket name is incorrect in the DDL	Check exact bucket name in IBM Cloud Console → Cloud Object Storage

9.3 Getting help

For issues with the TechZone environment (VM inaccessible, environment expired, etc.):

44. Raise a support ticket at https://techzone.ibm.com/support
45. Reference Lab 2257 and include your reservation ID (shown on the reservation page).
46. For IBM watsonx.data documentation: https://www.ibm.com/docs/en/watsonx/watsonx-data
47. For Confluent Platform documentation: https://docs.confluent.io/platform/7.6/

IBM TechXchange 2026 / © 2026 IBM Corporation

10 Lab Reference Materials

The following resources are available for students to continue their education after the lab. All links are publicly accessible.

10.1 IBM watsonx.data

- Product page: https://www.ibm.com/products/watsonx-data
- Documentation: https://www.ibm.com/docs/en/watsonx/watsonx-data
- TechZone trial environment: https://techzone.ibm.com
- Apache Iceberg specification: https://iceberg.apache.org/spec/

10.2 Confluent Platform 7.6

- Platform documentation: https://docs.confluent.io/platform/7.6/
- ksqlDB documentation: https://docs.ksqldb.io/
- Schema Registry API reference: https://docs.confluent.io/platform/current/schema-registry/develop/api.html
- Apache Flink on Confluent: https://docs.confluent.io/platform/current/flink/

10.3 Lab source code and automation

- Post-deploy Ansible playbook: realtime-fraud-detection-lab/confluent-wxdata-fraud-lab-full.yml
- Synthetic transaction producer: realtime-fraud-detection-lab/files/transaction_producer.py
- Lab verification script: realtime-fraud-detection-lab/files/verify_lab_setup.sh
- watsonx.data connection template: realtime-fraud-detection-lab/files/wxdata-connection.json
- Fraud 360 Command Center dashboard: fraud-detection-dashboard.html
- TechZone environment: https://techzone.ibm.com (search 'Lab 2257 Fraud Detection')

10.4 TechXchange 2026

- Session catalogue and lab downloads: https://techxchange.ibm.com